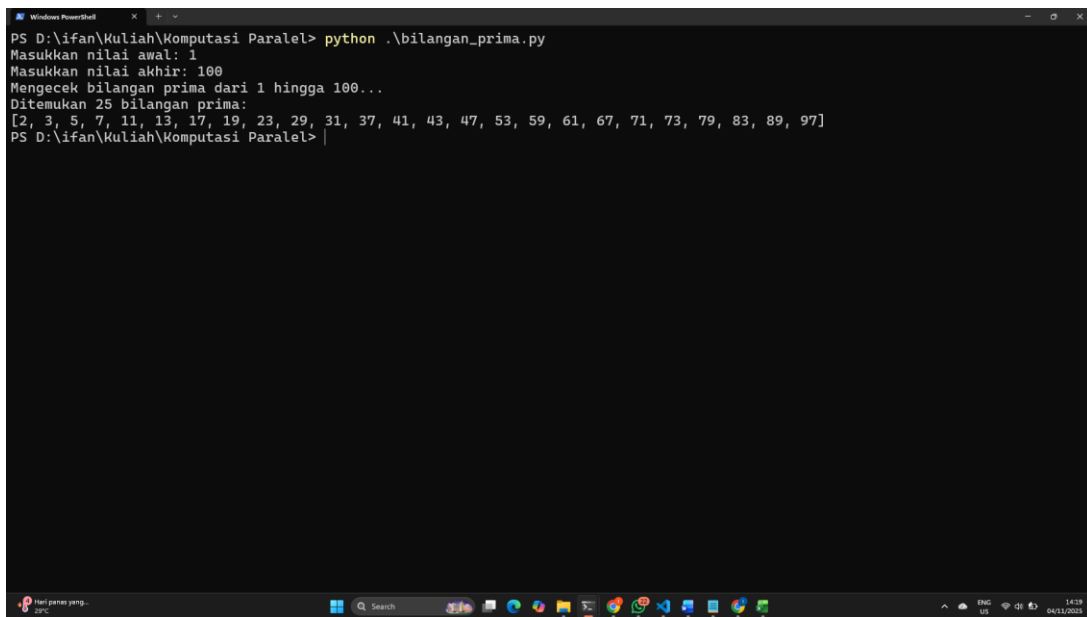


Nama : Ifan Prima Nursaid
NIM : 231351063
Kelas : Malam A

Kode sumber (source code) untuk semua program yang dibahas dalam laporan ini dapat diakses secara publik melalui repositori GitHub pribadi saya pada tautan berikut:

<https://github.com/ifanprima130206/Tugas-Komputasi-Paralel>

1. Tugas 1 membuat aplikasi komputasi paralel menentukan bilangan prima



```
PS D:\ifan\Kuliah\Komputasi Paralel> python .\bilangan_prima.py
Masukkan nilai awal: 1
Masukkan nilai akhir: 100
Mengecek bilangan prima dari 1 hingga 100...
Ditemukan 25 bilangan prima:
[2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71, 73, 79, 83, 89, 97]
PS D:\ifan\Kuliah\Komputasi Paralel>
```

Foto ini adalah tangkapan layar (screenshot) dari Windows PowerShell. Ini menunjukkan hasil setelah program Python `bilangan_prima.py` dijalankan.

- Input: Pengguna memasukkan 1 untuk "nilai awal" dan 100 untuk "nilai akhir".
- Proses: Program kemudian mencetak pesan "Mengecek bilangan prima dari 1 hingga 100...".
- Output: Program berhasil menemukan 25 bilangan prima di antara 1 dan 100, lalu menampilkannya dalam sebuah daftar (list).

```
from multiprocessing import Pool

def is_prime(n):
    if n < 2:
        return False
    for i in range(2, int(n**0.5) + 1):
        if n % i == 0:
            return False
    return True

def check_prime(n):
    return n if is_prime(n) else None

def input_range():
    while True:
        try:
            start = int(input("Masukkan nilai awal: "))
            end = int(input("Masukkan nilai akhir: "))
            if start < 0 or end < 0:
                print("Nilai harus ≥ 0, coba lagi.")
                continue
            if start > end:
                print("Nilai awal harus ≤ nilai akhir, coba lagi.")
                continue
            return start, end
        except ValueError:
            print("Input harus berupa angka, coba lagi.")

if __name__ == "__main__":
    start, end = input_range()
    numbers = range(start, end + 1)

    print(f"Mengecek bilangan prima dari {start} hingga {end}...")

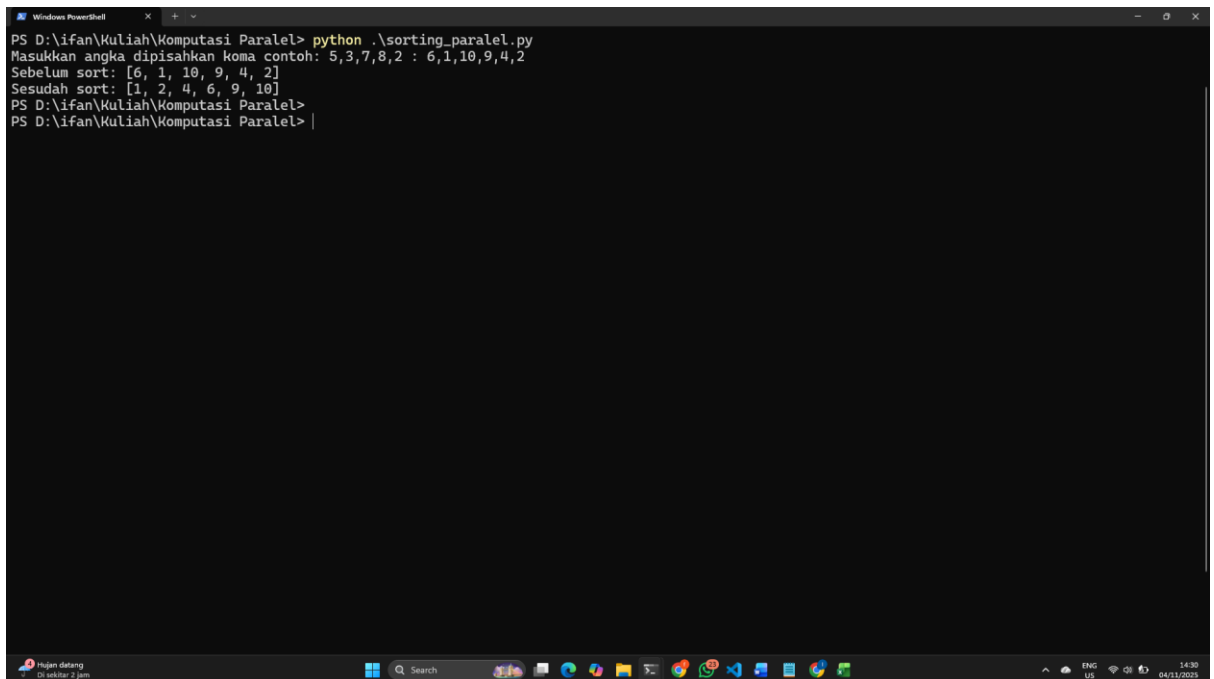
    with Pool() as pool:
        primes = list(filter(None, pool.map(check_prime, numbers)))

    print(f"Ditemukan {len(primes)} bilangan prima:")
    print(primes)
```

Foto ini adalah tangkapan layar dari editor kode Visual Studio Code yang menampilkan isi file `bilangan_prima.py`.

- **Pustaka (Library):** Kode ini mengimpor `Pool` dari modul `multiprocessing`. Ini adalah bagian kunci yang akan digunakan untuk menjalankan perhitungan secara paralel.
- **Fungsi:** Screenshot ini menunjukkan tiga fungsi yang telah dibuat:
 1. `is_prime(n)`: Fungsi utama untuk memeriksa apakah satu angka `n` adalah bilangan prima atau bukan.
 2. `check_prime(n)`: Fungsi pembantu (helper) yang akan mengembalikan angka `n` jika ia prima, dan `None` jika tidak. Fungsi seperti ini biasanya dipakai bersamaan dengan `Pool.map`.
 3. `input_range()`: Fungsi untuk meminta input "nilai awal" dan "nilai akhir" dari pengguna.

2. Tugas 2 membuat aplikasi komputasi paralel sorting



```
PS D:\ifan\Kuliah\Komputasi Paralel> python .\sorting_paralel.py
Masukkan angka dipisahkan koma contoh: 5,3,7,8,2 : 6,1,10,9,4,2
Sebelum sort: [6, 1, 10, 9, 4, 2]
Sesudah sort: [1, 2, 4, 6, 9, 10]
PS D:\ifan\Kuliah\Komputasi Paralel>
```

Ini adalah tangkapan layar dari Windows PowerShell yang menunjukkan hasil eksekusi program `sorting_paralel.py`.

- **Input:** Pengguna memasukkan deretan angka 6, 1, 10, 9, 4, 2 saat diminta.
- **Output:** Program kemudian mencetak dua baris:
 1. **"Sebelum sort:"** yang menunjukkan list angka asli sesuai input pengguna.
 2. **"Sesudah sort:"** yang menunjukkan list yang sama setelah berhasil diurutkan oleh algoritma paralel menjadi [1, 2, 4, 6, 9, 10].

```
Untitled

from multiprocessing import Pool

def merge(left, right):
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] < right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1
    result.extend(left[i:])
    result.extend(right[j:])
    return result

def parallel_merge_sort(arr):
    if len(arr) ≤ 1:
        return arr
    mid = len(arr) // 2
    left = arr[:mid]
    right = arr[mid:]

    with Pool(2) as pool:
        left_sorted, right_sorted = pool.map(sorted, [left, right])
    return merge(left_sorted, right_sorted)

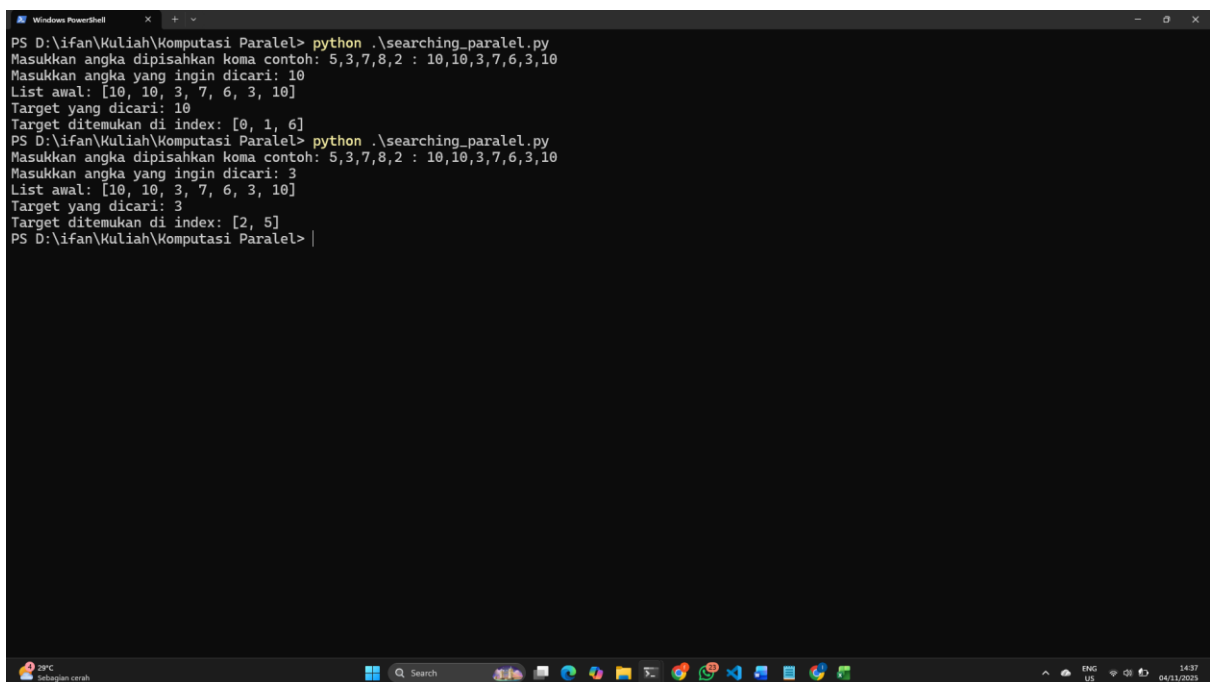
def input_numbers():
    while True:
        user_input = input("Masukkan angka dipisahkan koma contoh: 5,3,7,8,2 : ")
        try:
            numbers = [int(x.strip()) for x in user_input.split(",")]
            if not numbers:
                print("Daftar kosong, coba lagi.")
                continue
            return numbers
        except ValueError:
            print("Input harus berupa angka, coba lagi.")

if __name__ == "__main__":
    arr = input_numbers()
    print("Sebelum sort:", arr)
    sorted_arr = parallel_merge_sort(arr)
    print("Sesudah sort:", sorted_arr)
```

Ini adalah tangkapan layar dari editor Visual Studio Code yang menampilkan program Python untuk **sorting paralel**.

- **Metode:** Kode ini mengimplementasikan algoritma **Merge Sort** yang dimodifikasi untuk berjalan secara paralel.
- **Fungsi Utama:**
 - `merge(left, right)`: Fungsi standar dari merge sort yang menggabungkan dua sub-array (left dan right) yang sudah terurut menjadi satu array terurut.
 - `parallel_merge_sort(arr)`: Ini adalah inti dari logika paralelnya.
 1. Ia membagi array (arr) menjadi dua bagian, left dan right.
 2. Ia menggunakan `with Pool() as pool`: untuk membuat *pool* proses.
 3. Baris `pool.map(sorted, [left, right])` adalah bagian paralelnya. Ini akan menjalankan fungsi `sorted` (fungsi *built-in* Python) pada left dan right **secara bersamaan** di proses yang berbeda.
 4. Setelah kedua bagian selesai diurutkan secara paralel, hasilnya digabungkan kembali menggunakan fungsi `merge`.
- **Input/Output:** Kode ini juga memiliki fungsi `input_numbers()` untuk menerima masukan dari pengguna dan blok `if __name__ == "__main__":` untuk menjalankan program, mencetak list sebelum dan sesudah di-sortir.

3. Tugas 3 membuat aplikasi komputasi paralel searching



```
PS D:\ifan\Kuliah\Komputasi Paralel> python .\searching_paralel.py
Masukkan angka dipisahkan koma contoh: 5,3,7,8,2 : 10,10,3,7,6,3,10
Masukkan angka yang ingin dicari: 10
List awal: [10, 10, 3, 7, 6, 3, 10]
Target yang dicari: 10
Target ditemukan di index: [0, 1, 6]
PS D:\ifan\Kuliah\Komputasi Paralel> python .\searching_paralel.py
Masukkan angka dipisahkan koma contoh: 5,3,7,8,2 : 10,10,3,7,6,3,10
Masukkan angka yang ingin dicari: 3
List awal: [10, 10, 3, 7, 6, 3, 10]
Target yang dicari: 3
Target ditemukan di index: [2, 5]
PS D:\ifan\Kuliah\Komputasi Paralel> |
```

Ini adalah tangkapan layar dari Windows PowerShell yang menunjukkan program `searching_paralel.py` dijalankan dua kali dengan sukses.

- **Eksekusi Pertama:**

- **Input List:** 10, 10, 3, 7, 6, 3, 10
- **Target:** 10
- **Output:** Program berhasil menemukan bahwa angka 10 ada di 3 lokasi dan mencetak: "Target ditemukan di index: [0, 1, 6]".

- **Eksekusi Kedua:**

- **Input List:** (Daftar yang sama)
- **Target:** 3
- **Output:** Program berhasil menemukan bahwa angka 3 ada di 2 lokasi dan mencetak: "Target ditemukan di index: [2, 5]".

```
from multiprocessing import Pool

def find_index(args):
    arr, target, offset = args
    return [i + offset for i, x in enumerate(arr) if x == target]

def input_numbers():
    while True:
        user_input = input("Masukkan angka dipisahkan koma contoh: 5,3,7,8,2 : ")
        try:
            numbers = [int(x.strip()) for x in user_input.split(",")]
            if not numbers:
                print("Daftar kosong, coba lagi.")
                continue
            return numbers
        except ValueError:
            print("Input harus berupa angka, coba lagi.")

def input_target():
    while True:
        try:
            target = int(input("Masukkan angka yang ingin dicari: "))
            return target
        except ValueError:
            print("Input harus berupa angka, coba lagi.")

if __name__ == "__main__":
    arr = input_numbers()
    target = input_target()

    n_process = 4
    chunk_size = len(arr) // n_process
    chunks = []
    for i in range(n_process):
        start = i * chunk_size
        end = None if i == n_process - 1 else (i + 1) * chunk_size
        chunks.append((arr[start:end], target, start))

    with Pool(n_process) as pool:
        results = pool.map(find_index, chunks)

    indices = [idx for sublist in results for idx in sublist]

    print(f"List awal: {arr}")
    print(f"Target yang dicari: {target}")
    print(f"Target ditemukan di index: {indices}")
```

Ini adalah tangkapan layar editor kode yang menampilkan program Python untuk **pencarian paralel**.

- **Logika:** Program ini tidak menghentikan pencarian saat menemukan target pertama, melainkan **mencari semua kemunculan** (find all) target di dalam daftar.
- **Pembagian Tugas:**
 1. Di dalam blok `if __name__ == "__main__":`, kode ini membagi daftar angka yang besar (`arr`) menjadi 4 bagian lebih kecil (`chunks`).
 2. Setiap "chunk" ini dibundel bersama dengan nilai target yang dicari dan start (indeks awal chunk tersebut) sebagai argumen.
- **Fungsi Paralel:**
 - `find_index(args)`: Ini adalah fungsi yang akan dijalankan secara paralel. Setiap proses menerima satu "chunk", target, dan offset (indeks awal). Fungsi ini mencari target di dalam chunk kecil tersebut dan mengembalikan daftar indeks di mana target ditemukan. Pentingnya, ia menambahkan offset pada setiap indeks yang ditemukan sehingga indeksnya sesuai dengan posisi di daftar *asli*.
- **Eksekusi:**
 - `pool.map(find_index, chunks)`: Ini adalah baris yang mengeksekusi pencarian. Keempat proses berjalan **bersamaan**, masing-masing mencari di "chunk" yang ditugaskan padanya.
 - Hasilnya (`results`) adalah daftar yang berisi daftar (list of lists). Kode ini kemudian "meratakannya" (`flatten`) menjadi satu daftar indices yang berisi semua indeks tempat target ditemukan di seluruh daftar asli.